

Product Requirements Document

AgentDock

The collaboration layer for AI-assisted software engineering

Field	Value
Version	4.0 — merges v2.0 (specification) with v3.0 (positioning)
Status	Draft. Buildable. Pending Phase 0 and rename.
Date	August 2026
Working name	AgentDock — BLOCKED, unresolved since v2.0 (\$3)
Owner	TBD
Supersedes	v1.0, v2.0, v3.0
Decision required	Direction call after Phase 0 (\$13)

Contents

1. Executive Summary
2. Vision and Positioning
3. Naming — Still Blocking
4. Problem and Market
5. Users
6. Hard Constraints
7. Goals and Non-Goals
8. Scope
9. Feature Specifications
10. CLI Surface
11. Architecture
12. Non-Functional Requirements
13. Validation and Roadmap
14. Risks
15. Open Source and Business Model
16. Open Questions
- A. Disposition of v3.0 Proposals
- B. What v4.0 Restores From v2.0

1. Executive Summary

AgentDock is a terminal-native collaboration layer for teams and individuals who use more than one AI coding agent. It coordinates work across the agents a developer already pays for, gives every agent the same project context, and records what each one did and why — so that reasoning survives the session it was produced in.

1.1 What v4.0 is

v3.0 established the right identity: this product is not another orchestrator, it is a memory and accountability layer for AI-assisted engineering. v2.0 established the constraints, scope discipline and specification detail required to build it. v4.0 takes the positioning from v3.0, the specification from v2.0, and rejects the proposals from v3.0 that are unbuildable, unvalidated or scope-fatal.

Source	Carried into v4.0	Dropped
v3.0	"Git for AI collaboration" identity; three-pillar structure; provenance as the headline capability; the explain/timeline command vocabulary; structured event ledger; AGENTS.md alongside .agentdock; softened Phase 0 gate	Nine-role agent org; auto-built knowledge graph in v1; event bus described as agent-subscribable; nine-feature MVP; Phase 4 enterprise dashboards
v2.0	Vendor-terms constraint; concurrency and failure model; security model; adapter contract; measurable metrics; four-feature MVP; risks and open questions	Binary Phase 0 kill gate; "orchestration tool" framing; blame as a command name

1.2 The three pillars

Pillar	What it means	v1 status
1. Orchestration	Run agents against one codebase without collision; hand work between them without re-explaining	Ships in v1 (minimum viable form)
2. Persistent project intelligence	Shared memory, decisions, rejected alternatives and provenance that outlive any session	Ships in v1 — this is the differentiator
3. Team operating model	Stable roles independent of which model fills them	Two roles in v1; expansion gated on evidence

These pillars are not equal in v1. Pillar 2 is the reason to build the product; pillars 1 and 3 exist in v1 only to the extent required to make pillar 2 real.

1.3 The one-sentence test

If a developer removes AgentDock after six months, what do they lose that they cannot get back? Answer: the record of why the code is the way it is. Every scope decision in this document is measured against that sentence.

2. Vision and Positioning

2.1 Vision

Git made code history durable and attributable. AI-assisted development has broken that: the code is versioned, but the reasoning that produced it — the instruction, the alternatives, the rejection, the review — evaporates when the terminal closes. AgentDock makes that layer durable and attributable too.

2.2 Positioning statement

*For developers who run two or more AI coding agents, AgentDock is the **collaboration layer for AI-assisted software engineering**. Unlike **session managers and agent kanban boards**, which coordinate **processes** and forget everything when the process ends, AgentDock coordinates **knowledge** and keeps it in the repository.*

2.3 The capability that defines the product

Six months after a change is merged, a developer asks why a caching layer exists. AgentDock answers:

```
$ agentdock explain src/cache/redis_layer.py:14

Implemented by  claude-code (role: developer), session 4f21a9
Task           #184 "Reduce p95 response latency on /feed"
Alternatives    in-memory LRU, CDN edge cache, Redis
Rejected        in-memory LRU — lost on deploy, no cross-instance sharing
                  CDN — response is user-specific, not cacheable at edge
Reviewed by     codex (role: reviewer) — 2 findings, both addressed
Merged by       anish · 2026-08-14 · commit a91c33f
```

No tool in the category does this today. Session-history browsers can search transcripts; none tie reasoning to lines of code. This is the wedge.

2.4 What the product is not called

Drop "Kubernetes for AI agents" (v1.0) and "Operating System" (v3.0). Both promise scheduling, resource management and self-healing that this product will not deliver, and both signal operational weight to an audience that wants a fast CLI. "Collaboration layer" is accurate and sets a promise the product can keep.

3. Naming — Still Blocking

Raised in v2.0 §3.4, unresolved in v3.0. No public artefact — repository, domain, package, post — should be created under the current name.

Conflict	Detail	Impact
AgentDock	Existing MIT-licensed AI agent framework at agentdock.ai and github.com/AgentDock/AgentDock, with a commercial Pro tier and its own knowledge-graph product	Domain, GitHub org, PyPI/npm namespace and search results are taken. Trademark exposure. Their graph product overlaps the v3.0 knowledge-graph proposal directly.
agent-deck	Active MIT terminal session manager for	Same category, near-identical name,

Conflict	Detail	Impact
	AI coding agents (Claude, Gemini, Codex, OpenCode)	guaranteed confusion

Selection criteria: free on PyPI, npm and GitHub; .dev or .sh domain available; no existing agent-framework collision; works as a CLI verb; not in the dock/deck/hub/orchestra family, which is saturated. Bias toward the ledger/record/provenance semantic field, since that is the product. Resolve before Phase 1.

4. Problem and Market

4.1 Problem statement

Developers now routinely run two or more coding agents on the same codebase. Each agent starts blind, so architecture and conventions are re-explained to every tool. Work moved between agents is transferred by copy-paste. Parallel agents collide on the same files. And the reasoning behind agent-written code — what was tried, what was rejected, what a reviewer flagged — exists only in a terminal scrollbar that is gone by the next morning.

The first three problems are commodity in 2026 and are addressed by a dozen tools. The fourth is not addressed by any of them.

4.2 Competitive landscape (August 2026)

Tool	Shape	Where AgentDock differs
Vibe Kanban	Cross-platform CLI + web kanban over Claude Code, Codex, Gemini, Amp, Cursor CLI; Apache-2.0; parent company Bloop shut down in 2026, now community-maintained	Board-shaped, session-scoped, no durable reasoning record. Its loss of commercial backing is the clearest opening in the category.
Conductor	macOS desktop; parallel Claude Code + Codex in isolated worktrees; free	macOS-only; process coordination, no memory layer
Claude Squad / Nimbalyt / Emdash	Local open-source parallel session managers over worktrees	Same — isolation and visibility, nothing persistent
agent-deck	One TUI for Claude/Gemini/Codex/OpenCode; worktree-aware; MCP; MIT	Closest surface overlap, and the name collides
CLI Agent Orchestrator (AWS)	Hierarchical multi-agent delegation over tmux	Already occupies the role-hierarchy idea. Do not compete here.
OpenHands / Sculptor	Platform-grade, containerised, self-hostable	Heavier deployment model; different buyer
tmux + AGENTS.md	The zero-install default many senior engineers actually use	The real competitor. v1 must beat three terminal tabs decisively, or it will not be adopted.

4.3 Strategic reading

- Session orchestration is saturated and has already produced commercial failures (Bloop/Vibe Kanban, Terragon). Do not enter on that axis.
- Durable cross-agent memory and provenance is genuinely open, and it compounds: the tool gets more valuable the longer a repository is worked on, which is the only real moat available here.
- Being maintained, cross-platform and vendor-neutral is now a differentiator in its own right, given the state of the incumbents.

5. Users

5.1 Primary persona — the multi-agent operator

- Runs two or more agent CLIs daily; typically Claude Code plus Codex or Gemini.
- Already improvised a worktree, tmux or tab-based workflow, and has abandoned at least one orchestrator.
- Pays for at least one subscription; cost-aware; cannot easily justify API spend layered on top.
- Adopts via one install command; abandons within a week if the tool adds friction.

5.2 Jobs to be done

Job	Trigger	Success
Recover the reasoning behind existing code	PR review, debugging, onboarding, incident	One command returns agent, instruction, alternatives, rejection, reviewer, merge
Hand a task to another agent without re-explaining	Agent A is blocked, or the task needs a different strength	Agent B is productive in under 60 seconds with no copy-paste
Get a second opinion on a diff	Before merging non-trivial agent-written code	A reviewing agent returns findings against a known diff and known conventions
Run work in parallel without collisions	Multiple independent tasks	Each agent isolated; merges explicit and reviewable

5.3 Deferred segments

Researchers, DevOps and Enterprise teams are out of scope for v1. Enterprise implies SSO, retention policy, centralised governance and compliance audit; naming the segment creates requirements the roadmap cannot absorb. v3.0 reintroduced enterprise dashboards at Phase 4 — removed again here.

6. Hard Constraints

These determine whether the product can exist. They are not negotiable by roadmap.

6.1 Vendor authentication and terms

During 2026, Anthropic clarified and enforced that subscription OAuth tokens (Free, Pro, Max) may be used only by Claude Code and Claude.ai. Third-party tools that extracted or replayed those tokens were blocked server-

side; several prominent projects removed built-in Anthropic authentication in response, and some users had accounts suspended. Other vendors have moved similarly.

Binding design rules:

- AgentDock invokes the vendor's official CLI binary as a child process. It never reads, stores, forwards, refreshes or proxies a credential.
- AgentDock never presents itself as, or spoofs, a vendor harness.
- User-supplied API keys are passed through the environment to the child process only, and are never persisted by AgentDock.
- Vendor rate limits, weekly caps and terms apply unchanged. Documentation must state plainly that parallel agents consume quota faster.
- Every adapter is a standing maintenance liability, not a one-time integration.

6.2 What can and cannot be reduced

AgentDock cannot share a KV cache, prompt cache or conversation state across vendor processes; each CLI maintains its own context window. Running three agents on one task costs roughly three times one agent. The v1.0 goal "reduce duplicated token usage" remains removed.

The accurate and achievable claim, adopted from v3.0: minimise redundant context reconstruction across agents through structured handoffs. If agent A has already explored 400 files, agent B should receive A's conclusions rather than repeat the exploration. That reduces B's work; it does not reduce A's, and it does not merge their contexts. This is measurable (§7.1) and should be measured rather than asserted.

6.3 Agents cannot collaborate in real time

Vendor CLIs are independent processes with no inter-agent protocol. Collaboration is a sequential relay with parallel isolation: A finishes, AgentDock captures and structures the output, B is started with a constructed prompt. This constrains §9.6 directly — see the correction to the v3.0 event bus.

7. Goals and Non-Goals

7.1 Goals with targets

v3.0 lists goals without numbers ("reduce context reconstruction time", "increase developer trust"). Restored to measurable form:

Goal	Metric	v1 target
Make agent work explainable	% of committed agent changes that resolve to instruction + session + alternatives	Over 90%
Minimise redundant context reconstruction	Tokens consumed by the receiving agent, handoff vs cold start, on matched tasks	40% reduction, median
Eliminate re-explanation on handoff	Median time from "A done" to "B productively working"	Under 60s, zero copy-paste
Prevent parallel collisions	Clobbering incidents per active user per month	Under 0.1

Goal	Metric	v1 target
Stay out of the way	Install to first successful handoff	Under 10 minutes
Cost visibility	% of sessions with captured usage data where the vendor CLI exposes it	Over 80%
Retention (the real one)	Users still active 30 days after first handoff	Over 30%

7.2 Non-goals

- Not an LLM, IDE, Git replacement or inference platform.
- Not a proxy, gateway or credential broker for any provider.
- Not autonomous — no agent starts without an explicit human instruction in v1.
- Not hosted or multi-tenant in v1; local-first, single user, no telemetry by default.
- Not a competitor to AGENTS.md — AgentDock consumes and extends the open standard.
- Not a simulated software company. See §8.3.

8. Scope

8.1 v1 scope

Feature	Priority	Rationale
Provenance ledger (F3)	Must	The product. Everything else exists to feed it.
Project memory: AGENTS.md + decision log (F1)	Must	The input to provenance and to handoffs
Structured handoff (F2)	Must	The mechanism that generates most memory, and the measurable win
Two adapters: Claude Code + one other (F0)	Must	Two providers proves the abstraction; more is padding
Worktree isolation (F4)	Should	Table stakes; wrap git plumbing thinly, do not innovate
Two roles: developer, reviewer (F5)	Should	Enough to make review real; see §8.3 on expansion
Review workflow	v1.1	Natural extension of handoff once briefs are proven
Session resume	v1.1	Bounded by each vendor CLI's own resume support
Cost tracking	v1.1	Depends on adapters exposing usage
Command router	Later	Only meaningful once there are enough

Feature	Priority	Rationale
		roles to route between
Compare mode	Later	Triples cost for unvalidated benefit. Gate on demand evidence.
Knowledge graph	Later	See §8.3
TUI dashboard	Later	Every competitor has one. Differentiate on memory, then on surface.
Plugin SDK / marketplace	Later	Adapters stay in-tree until there is a contributor base
VS Code extension, enterprise features	Not planned	Different products; do not start while the CLI is unproven

8.2 Scope discipline note

v1.0 proposed ten MVP features. v2.0 cut to four. v3.0 returned to nine — agent registry, command router, worktrees, handoffs, memory, review pipeline, compare mode, audit log and plugin SDK foundation — reinstating two items that had been cut for cost and validation reasons. v4.0 returns to four, with a documented promotion path. Every reinstatement should require evidence, not enthusiasm.

8.3 Rejected: the simulated software company

v3.0 proposes a nine-role hierarchy (CEO, Engineering Manager, Architect, Developer, Reviewer, Tester, Security, Performance, Documentation), each backed by any provider. This is rejected for v1 and probably permanently, on evidence:

- Role-hierarchy systems have been built repeatedly since 2023 (ChatDev, MetaGPT, AgileCoder, and successors). Published evaluations found ChatDev unable to autonomously produce a working Tetris game except on the tenth attempt — a toy relative to a production repository.
- Cost scales with the hierarchy. Large agent groups were measured at over \$10 per HumanEval task from serial inter-agent messaging alone. Nine roles against a real codebase is not a viable per-task cost for the target user, who is already quota-constrained (\$6.1).
- Error propagation is the dominant failure mode: a weak reviewer or tester role passes incorrect code and every downstream role builds on it. More roles means more places to compound an error, not more quality.
- AWS's CLI Agent Orchestrator already ships hierarchical role delegation. Entering there means competing on someone else's ground with no advantage.

Position instead: roles are a usability layer, capped by default at two or three active roles per task, with expansion driven by measured outcome quality rather than organisational metaphor. A role is a saved prompt prefix plus a default provider plus a permission scope — useful, but a user can approximate it with shell aliases, so it cannot be the differentiator.

8.4 Deferred: automatic knowledge graph

v3.0 proposes automatically extracting entities, relationships and implementation history into a queryable graph. Deferred, not rejected, for four reasons: it is the hardest item on the list; auto-extracted graphs drift from the code and become confidently wrong; agents with large context windows plus grep are increasingly good at answering structural questions directly; and it sits in direct tension with the context-bloat evidence in §9.1. The

decision log delivers most of the value at a fraction of the cost. Revisit only if Phase 2 shows users asking structural questions the log cannot answer.

9. Feature Specifications

9.1 F1 — Project memory

Two stores, deliberately separated. v3.0 is correct that both are needed; the boundary matters.

Store	Contents	Property
AGENTS.md (repo root)	Static conventions, build/test commands, constraints. Human-authored.	Portable — works for any agent, with or without AgentDock installed
.agentdock/	decisions.md (append-only), ledger.jsonl, index.db, sessions/, handoffs/, config/	Machine state — AgentDock-specific, but plain-text where it holds reasoning

- AGENTS.md is never rewritten by AgentDock without explicit confirmation.
- decisions.md is append-only Markdown, git-committed, readable without any tooling. If AgentDock disappears, the reasoning survives in the repo.
- index.db (SQLite) is a rebuildable index over ledger.jsonl, never the source of truth.

Context budget — a hard requirement, not a preference

Published research on agent context files found that oversized or machine-generated context reduced task success rates by roughly 3% while raising inference cost over 20%, with long architecture sections specifically identified as harmful. An unbounded memory layer makes agents worse. Therefore:

- Hard token budget per injected brief — default 2,000 tokens, configurable, enforced.
- Decisions ranked by recency and file-path relevance; the full log is never injected.
- agentdock memory --preview shows exactly what will be injected before it is.
- Measured against a no-context control during Phase 0 and at every release.

9.2 F2 — Structured handoff

A handoff is a structured artefact, not a transcript. Transcripts are long, noisy and expensive to inject.

```
goal:          one line — what was being attempted
status:        complete | blocked | partial
files_touched: [path, one-line description of the change]
decisions:     [decision, rationale]
alternatives:  [option, why not]           # highest-value field
open_questions: [what the next agent must resolve]
verification:  commands run, and their results
```

- agentdock handoff --to codex --role reviewer composes the brief, shows it for confirmation, then launches the target CLI with it.
- The brief is produced by asking the outgoing agent to summarise its own session against the schema. Where a vendor CLI cannot be queried non-interactively, fall back to a template the user edits.
- Every handoff writes to the ledger and appends any decisions to decisions.md.

- Acceptance: a decision recorded in a Claude Code session appears in the brief given to a Codex session touching the same files, inside the token budget.

Primary risk: if a generated brief is worse than three sentences the developer would have typed, nothing downstream matters. This is the first thing Phase 0 tests.

9.3 F3 — Provenance ledger

The headline capability. Append-only JSONL as source of truth, SQLite index for query speed.

- Records per event: id, timestamp, session, agent, model where exposed, role, instruction, files changed, commit SHAs, usage/cost where the CLI reports it.
- agentdock explain <file>[:line] returns the §2.3 output — agent, task, alternatives, rejection, reviewer, merge.
- agentdock timeline renders the project's event history, filterable by file, agent, role or date.
- File contents are never recorded by default — paths and hashes only. Content capture is opt-in.
- Ledger is git-committed by default (configurable), so provenance travels with the repository and survives machine changes.

9.4 F4 — Isolation, concurrency and failure

Unspecified in both v1.0 and v3.0. This is where multi-agent tools break in practice.

Situation	v1 behaviour
Two agents assigned overlapping files	Refuse; require worktree isolation, or explicit --force with a warning
Agent modifies files outside its declared scope	Detect at session end, flag in the ledger, surface in review
Conflicting changes across worktrees	No auto-merge in v1. Present the conflict; hand to the human or to a merge role.
Agent crashes or is killed mid-session	Mark session incomplete, preserve the worktree, generate a partial handoff from what is known
Vendor rate limit hit	Surface the vendor error verbatim, mark the session blocked, never silently retry against another provider
Ledger and worktree disagree	Ledger is advisory, git is authoritative. Never let AgentDock state override repository state.

9.5 F5 — Roles

- v1 ships two roles: developer and reviewer. Each is a prompt prefix, a default provider, a permission scope and a handoff schema variant.
- Roles are declared in .agentdock/config/roles.yaml and are user-editable.
- Provider swap must not change the workflow — this is the one part of the v3.0 role vision worth keeping.
- Expansion to architect, tester, security beyond v1 requires measured evidence that the added role improves outcome quality, not just that it is nameable (§8.3).

9.6 F6 — Event ledger (the "reasoning bus", corrected)

v3.0 proposes an event stream that "every agent subscribes to". Agents cannot subscribe to anything: vendor CLIs are subprocesses with no inbound channel and no persistent runtime (§6.3). Describing it as a bus will mislead contributors and produce an architecture that cannot be implemented.

Corrected design: a single append-only event log that AgentDock writes and selectively replays into prompts. Same events, honest mechanics.

TaskCreated · SessionStarted · DecisionRecorded · HandoffCreated
SessionEnded · ReviewRequested · ReviewCompleted · MergeCompleted

- The log is the ledger (F3) — one store, not two. Do not build a separate bus.
- Replay is filtered and budgeted (§9.1); an agent receives the events relevant to its task, never the whole stream.
- A future daemon could offer real subscription for AgentDock-native tooling. Out of scope for v1, and it would not make vendor CLIs subscribers.

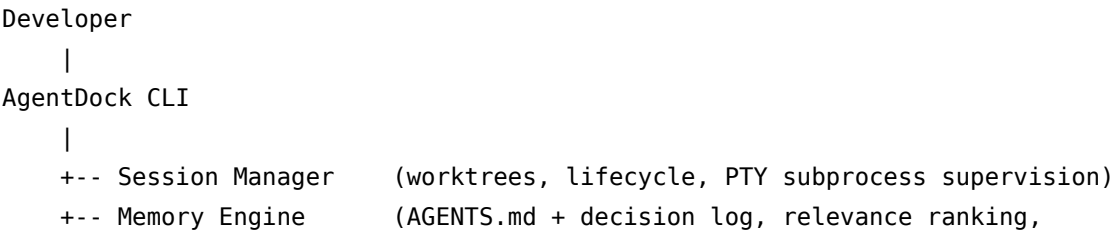
10. CLI Surface

Eight commands for v1. v3.0's explain and timeline are better names than v2.0's blame and log, and are adopted.

Command	Purpose
agentdock init	Detect installed agent CLIs, create or extend AGENTS.md, scaffold .agentdock/
agentdock run <role> "<task>"	Start an agent session in an isolated worktree
agentdock handoff --to <agent> [--role <role>]	Compose a brief, confirm, launch the next agent
agentdock explain <file>[:line]	Why does this code exist — agent, instruction, alternatives, reviewer, merge
agentdock timeline [--file --agent --since]	Project event history
agentdock memory [--preview --add --prune]	Inspect, edit and preview injected context
agentdock status	Live sessions, worktrees, blocked agents, degraded adapters
agentdock merge <session>	Review a worktree diff and merge

- Every command must be scriptable non-interactively; --json on explain, timeline, memory and status.
- No daemon required in v1.
- start, stop, resume, ask, agents, review, compare, config, update, knowledge and assign are deferred or folded in above.

11. Architecture



```

|                                     budgeted brief composition)
+-- Provenance Ledger (append-only JSONL, SQLite index, explain/timeline)
|
Adapter Layer  - subprocess only, zero credential handling
|
claude  |  codex  |  gemini  |  ollama-backed CLI

```

The Memory Engine and Provenance Ledger are peers of the Session Manager, not layers beneath a router. v1.0 and v3.0 both place an orchestrator or router between the user and everything else, which forces all value through a component that is not needed until there are many roles to route between.

11.1 Adapter contract

Vendor CLIs differ enough that a naive common interface will leak. Each adapter declares and is tested against:

```

invoke(prompt, cwd, env)           -> streamed output, exit code
supports_noninteractive: bool
supports_resume: bool
reports_usage: bool                # token/cost extraction where available
summarise_session(schema)          -> handoff brief, or null if unsupported

```

- A conformance suite runs against every adapter each release. Failing adapters are marked degraded in agentdock status rather than misbehaving silently.
- Tested vendor CLI versions are pinned in documentation.

11.2 Technology

Layer	Choice	Note
Language	Python	Fast to build in. Distribution is the tax: ship via uv/uvx or a single binary, never "pip install and hope". The tools with traction in this category are Rust or TypeScript, largely for this reason — revisit if adoption stalls on install friction.
Subprocess control	pexpect / ptyprocess	The hardest part of the build: PTY handling, streaming capture, ANSI stripping, timeouts, clean kill. Budget accordingly.
Git	git CLI directly for worktrees; GitPython elsewhere	GitPython handles worktrees awkwardly
Storage	JSONL source of truth + SQLite index	Diffable, git-friendly, survives schema change
Output	Rich	Formatting only in v1; no TUI dashboard
Concurrency	asyncio	Subprocess supervision dominates complexity, not async I/O

12. Non-Functional Requirements

12.1 Security

- No credential handling, ever (§6.1). Stated in the README, not buried.
- Permission-bypass flags (e.g. --dangerously-skip-permissions) are never set by default or globally; only per-session, explicitly, by the user.
- Unattended runs are supported only inside a worktree or container, and the docs must say so.
- Prompt injection: brief content may originate from repository files or agent output. Briefs are shown before injection, and injected content is fenced and labelled as untrusted data.
- The ledger holds instruction text and is git-committed by default. Documentation must warn against secrets in prompts, and a redaction hook must be available before the ledger is committed by default.

12.2 Performance

- Cold start under 200 ms for status, explain and timeline.
- Brief composition under 2 seconds excluding any model call.
- Five concurrent sessions on a laptop with AgentDock's own resident memory under 300 MB.
- explain must return in under 1 second on a repository with 50,000 ledger events.

12.3 Compatibility and privacy

- macOS and Linux at v1; Windows via WSL. Native Windows is a stated v1 non-goal — and an opening, given the leading desktop competitor is macOS-only.
- Degrade gracefully with a single agent installed: memory and provenance must be useful without any handoff.
- No telemetry in v1. If added later: opt-in, documented, off by default.
- All state local, inside the repository or a user-scoped directory.

13. Validation and Roadmap

13.1 Phase 0 — two weeks, directional not binary

v2.0 proposed a kill gate. v3.0 correctly argued this was too blunt: modest handoff gains do not invalidate the provenance thesis. Phase 0 is therefore a direction-setting experiment with one floor.

Questions to answer:

- Do structured handoffs reduce re-explanation time and receiving-agent token consumption, versus a cold start and versus a hand-written control?
- Do developers trust AI-generated summaries enough to act on them without re-reading the source session?
- Which parts of the workflow are worth automating, and which should stay manual?
- Which vendor CLIs support non-interactive invocation and self-summarisation, and which expose usage data?
- When shown the §2.3 explain output as a mockup, do target users say it would change their behaviour?

Interpreting the result

Outcome	Response
Handoffs strong, provenance interest strong	Build v1 as scoped
Handoffs modest, provenance interest strong	Rebalance: provenance ships first, handoff becomes one of several capture mechanisms alongside manual decision entry
Handoffs strong, provenance interest weak	Reconsider seriously — this is the crowded axis, and the product would be a late entrant on it
Both weak	Stop. The remaining product is session management, which a dozen maintained tools already do.

Floor: at least one of handoff or provenance must show a clear, articulable win with real users. That is the only hard gate.

13.2 Roadmap

Phase	Scope	Exit criteria
Phase 0 — Validate (2 wks)	Handoff experiment, explain mockup testing, 10 user interviews, adapter feasibility spikes, rename resolved	Floor met; two adapters proven non-interactive; name secured
Phase 1 — Core (8–10 wks)	Two adapters, ledger, memory, handoff, worktree isolation, two roles, eight commands	10 external users complete a handoff unaided; explain answers a real question on a real repo
Phase 2 — Depth (6–8 wks)	Review workflow, session resume, third adapter, cost tracking, --json everywhere, redaction hook	30% of Phase 1 users active at day 30
Phase 3 — Surface (open)	TUI, command router, role library, plugin SDK, compare mode and knowledge graph if demand is proven	Begins only if Phase 2 retention holds

v3.0's Phase 4 (enterprise collaboration and dashboards) is removed. A pre-validation project should not carry an enterprise phase in its roadmap; reviewers read it as evidence the plan is aspirational.

14. Risks

Risk	Severity	Mitigation
Vendor terms change again, or CLIs restrict subprocess wrapping	Critical	Subprocess-only, zero credentials, no harness spoofing. Thin replaceable adapters. Monitor vendor policy as a

Risk	Severity	Mitigation
		standing task.
Handoff briefs no better than manual	High	Phase 0. Fallback path: provenance-first product (§13.1).
Provenance is a "nice idea" nobody pays attention to	Critical	Test the explain mockup in Phase 0 before building. This risk is under-tested and is the real bet.
Vendor CLI changes break adapters	High	Conformance suite, visible degradation, pinned tested versions
Category saturation, no adoption	High	Do not launch as an orchestrator. Lead with explain.
Vendors ship native cross-agent memory	High	Stay vendor-neutral and standards-based; the value is precisely where vendors will not go — across competitors
Context injection degrades agent output	Medium	Hard budgets, relevance ranking, preview, measured against a no-context control
Ledger becomes noise nobody reads	Medium	explain is the read path, not the raw log. If explain is not used weekly, the feature has failed regardless of ledger completeness.
Scope re-inflation between drafts	Medium	Every reinstated feature requires written evidence of demand. This has already happened once (§8.2).
Solo maintainer burnout	Medium	Two adapters, in-tree only, no plugin SDK until contributors exist
No revenue model; category has seen shutdowns	Medium	Decide \$15 before Phase 1

15. Open Source and Business Model

v3.0 commits to MIT, a public roadmap, provider plugins and community governance. Directionally right; two decisions remain unmade and should be made before Phase 1, because they change scope.

- Licence: MIT maximises adoption; Apache-2.0 adds patent protection and is what several competitors chose. Recommend Apache-2.0 given the vendor-adjacent position.
- Portfolio project or business? If a business, the CLI is not the monetisable layer — team-shared memory and provenance across a repository is. That implies a hosted or sync component, which is explicitly out of v1 scope, so the decision must be made deliberately rather than discovered.
- Community governance is premature before there is a community. Public roadmap and clear contribution docs at Phase 1; governance structure at Phase 3.

- Do not accept adapter contributions until the conformance suite exists (§11.1), or adapter quality becomes unmanageable.

16. Open Questions

- Who generates the handoff brief — the outgoing agent (costs tokens, better quality) or local heuristics (free, likely worse)? Phase 0 answers this.
- Is the ledger committed to the repository by default? It enables team memory but adds diff noise and may leak instruction text.
- Does the memory and provenance layer work standalone, with no orchestration at all, as a smaller and faster first release? This may be the better v1.
- What is the single metric that determines whether the project continues after Phase 1?
- Portfolio project or business (§15)?
- What is the product called (§3)?

Appendix A — Disposition of v3.0 Proposals

v3.0 proposal	Disposition	Reason
"Git for AI collaboration" identity	Accepted	Sharper and more defensible than the v2.0 framing. Now the vision (§2.1).
Three pillars: orchestration, persistent intelligence, team operating model	Accepted, reweighted	Adopted as the product structure (§1.2), but the pillars are not equal in v1 — pillar 2 is the reason to build.
Provenance as the headline capability	Accepted and promoted	Moved from a supporting feature to the product's defining capability (§2.3, §9.3).
"Minimise redundant context reconstruction" wording	Accepted	More precise than v2.0's framing, and measurable. Now a goal with a target (§6.2, §7.1).
Keep .agentdock alongside AGENTS.md	Accepted	Correct — the tool needs machine state. v4.0 defines the boundary: AGENTS.md is portable context, .agentdock is machine state (§9.1).
explain, timeline command vocabulary	Accepted	Better names than v2.0's blame and log (§10).
Phase 0 should not be a binary kill gate	Accepted with a floor	Fair correction. Phase 0 is now direction-setting with one hard floor (§13.1).
Structured event stream (TaskCreated, ReviewCompleted, etc.)	Accepted, mechanics corrected	The events are right. "Every agent subscribes" is not implementable — vendor CLIs are subprocesses with no inbound channel. Rebuilt as a replayable

v3.0 proposal	Disposition	Reason
		log, and merged into the ledger rather than built as a second store (§9.6).
MIT licence, public roadmap, community governance	Partially accepted	Apache-2.0 recommended instead; governance deferred until a community exists (§15).
Nine-role software company (CEO through Documentation)	Rejected	Repeatedly built and repeatedly failed on real codebases; cost scales with hierarchy; error propagation compounds; AWS already occupies the ground (§8.3).
Automatic knowledge graph extraction	Deferred	Hardest item, drifts from code, competes with agents' own retrieval, and conflicts with the context-budget evidence. The decision log delivers most of the value (§8.4).
Nine-feature MVP	Rejected	Returns to v1.0 scope levels and reinstates compare mode and plugin SDK, both cut for cost and validation reasons (§8.1–8.2).
Phase 4 enterprise collaboration and dashboards	Removed	Creates obligations a pre-validation project cannot meet (§13.2).
Goals stated without targets	Rejected	Restored to measurable form with v1 numbers (§7.1).
"Operating System" framing	Rejected	Same failure as the Kubernetes analogy — promises scheduling and self-healing the product will not deliver (§2.4).

Appendix B — What v4.0 Restores From v2.0

These sections existed in v2.0, were absent from v3.0, and are required before implementation can start. Their absence is why v3.0 could be pitched from but not built from.

Section	Why it is required
§6.1 Vendor authentication constraint	Determines whether the product is permitted to exist at all
§7.1 Metrics with numeric targets	Without targets there is no way to know if v1 worked
§9.4 Concurrency, conflict and failure model	The most common practical failure point for multi-agent tools
§11.1 Adapter contract and conformance suite	Adapters are the standing maintenance cost; undefined contracts make that cost unbounded

Section	Why it is required
§12.1 Security and permissions	Permission-bypass defaults and prompt-injection surface are real exposure
§14 Risks	Several are severity-critical and change the plan
§16 Open questions	Six decisions remain unmade; three of them change scope
§3 Naming resolution	Raised in v2.0, still unresolved, still blocking

End of document.